

Especificação Arquitetural — Sistema de Gestão de Clientes

Reestruturação da aplicação legado no padrão MVC (Model-View-Controller) com foco no fluxo de autenticação e no controle de estado via sessão HTTP e cookies.

Documento:	Especificação Arquitetural (MVC + Gestão de Sessão)
Responsável:	Arquiteto de Software
Data:	Agosto de 2026
Referências:	Basham, Sierra & Bates — Head First Servlets & JSP; Kurniawan — Servlet & JSP: A Tutorial

1. Objetivo e Escopo

Este documento especifica a reestruturação arquitetural de uma aplicação legado do lado do servidor, migrando-a para o padrão MVC (Model-View-Controller). O escopo concentra-se no fluxo de autenticação do usuário — do envio das credenciais até o acesso a uma área restrita — detalhando a responsabilidade de cada camada (View, Controller e Model) e o mecanismo de controle de estado adotado pelo servidor (HttpSession e cookies), já que o protocolo HTTP é, por natureza, sem estado (stateless).

2. Visão Geral do Padrão MVC na Aplicação

A separação de responsabilidades proposta segue a organização clássica descrita por Kurniawan para aplicações Java EE baseadas em Servlets e JSP:

View (JSP/HTML): Camada de apresentação. Responsável exclusivamente por renderizar a interface — formulário de login, dashboard, tabela de clientes — sem conter lógica de negócio ou acesso direto a dados. Recebe do Controller apenas os objetos já processados (via request attributes).

Controller (Servlets/Filters): Recebe as requisições HTTP, interpreta os parâmetros enviados pela View, aciona os métodos apropriados do Model, decide qual View será exibida em seguida (forward ou redirect) e é o responsável por criar, ler e invalidar a sessão do usuário.

Model (Classes de domínio/DAO): Encapsula as regras de negócio e o acesso a dados (ex.: validação de credenciais, consulta ao repositório de clientes). Não conhece a View nem manipula objetos de request/response — mantendo o desacoplamento defendido por Basham como um dos principais benefícios do MVC.

Essa separação evita o antipadrão identificado na aplicação legado, em que páginas JSP continham código Java embutido (scriptlets) responsável tanto pela apresentação quanto pelo acesso ao banco de dados — dificultando manutenção e testes.

3. Diagrama do Fluxo MVC — Autenticação do Usuário

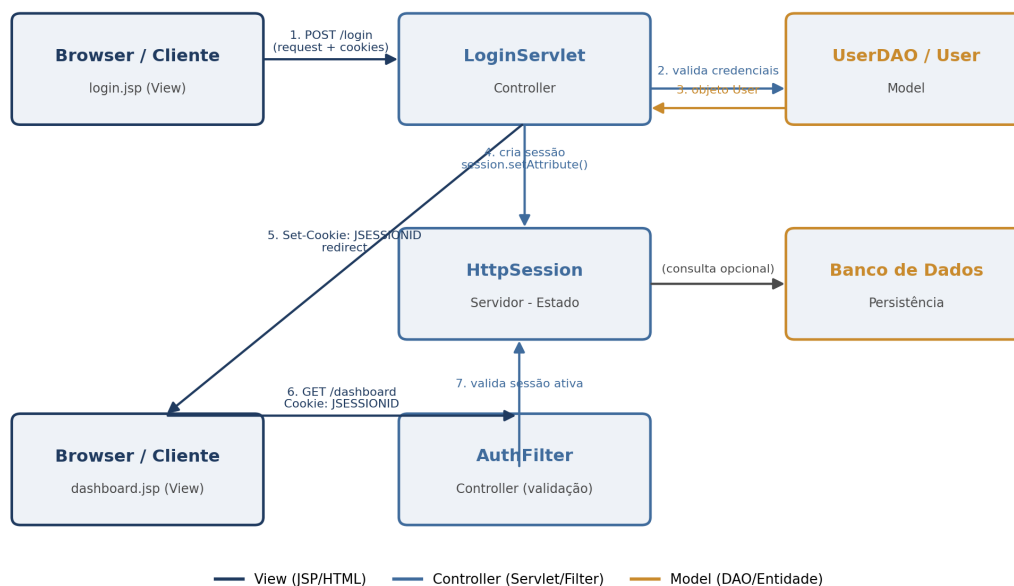


Figura 1 — Fluxo de requisição/resposta entre View, Controller e Model no processo de login e no acesso subsequente a uma página protegida.

3.1 Descrição passo a passo

- 1 O navegador envia um **POST** para `/login` com os dados preenchidos em `login.jsp` (a View).
- 2 O **LoginServlet** (Controller) lê os parâmetros da requisição e delega a validação ao Model, chamando `UserDAO.autenticar(usuario, senha)`.
- 3 O **Model** consulta a fonte de dados e retorna um objeto `User` válido ou `null`, sem qualquer conhecimento sobre a camada web.
- 4 Em caso de sucesso, o Controller cria uma **HttpSession** (`request.getSession(true)`) e armazena o usuário autenticado com `session.setAttribute("usuarioLogado", user)`.
- 5 O servidor devolve a resposta com o cabeçalho `Set-Cookie: JSESSIONID=...` e realiza um `redirect` para `dashboard.jsp`, evitando reenvio do formulário (padrão Post/Redirect/Get).
- 6 Nas requisições seguintes, o navegador reenvia automaticamente o cookie `JSESSIONID` no cabeçalho `Cookie`.
- 7 Um **AuthFilter** (Controller de infraestrutura) intercepta a requisição, valida se a sessão existe e contém o atributo de usuário; caso contrário, redireciona para a página de login.

4. Controle de Sessão e Cookies no Servidor

Como o protocolo HTTP não mantém estado entre requisições, cada chamada é tratada pelo servidor de forma independente. Para simular a continuidade da experiência do usuário autenticado, a especificação adota o mecanismo de **sessão HTTP** (interface `HttpSession` da API Servlet), apoiado por **cookies**, conforme descrito por Kurniawan no capítulo dedicado ao gerenciamento de sessão e por Basham no tratamento de estado em aplicações Servlet/JSP.

4.1 Criação e identificação da sessão

No primeiro acesso autenticado, o container Servlet gera um identificador único de sessão (Session ID) ao chamar `request.getSession(true)`. Esse identificador é transmitido ao navegador por meio do cookie `JSESSIONID`, incluído automaticamente pelo container no cabeçalho `Set-Cookie` da resposta — sem necessidade de manipulação manual pelo Controller.

```
HttpSession session = request.getSession(true);
session.setAttribute("usuarioLogado", user);
session.setMaxInactiveInterval(1800); // 30 minutos
```

4.2 Persistência do estado entre requisições

A cada nova requisição, o navegador reenvia o cookie `JSESSIONID`, permitindo que o container localize a instância de `HttpSession` correspondente no servidor e recupere os atributos armazenados (por exemplo, o

objeto do usuário logado). Esse é o mecanismo padrão de rastreamento de sessão (session tracking) descrito na especificação Servlet e detalhado tanto por Basham quanto por Kurniawan como a alternativa preferencial à reescrita de URL (URL rewriting), utilizada apenas como *fallback* quando o cliente tem cookies desabilitados.

4.3 Validação de acesso a páginas restritas

O `AuthFilter`, mapeado para os padrões de URL protegidos (ex.: `/clientes/*`, `/dashboard`), intercepta a requisição antes de qualquer Servlet ou View, verificando a existência da sessão e do atributo de autenticação:

```
HttpSession session = request.getSession(false);
boolean autenticado = (session != null
    && session.getAttribute("usuarioLogado") != null);

if (!autenticado) {
    response.sendRedirect("login.jsp");
    return;
}
chain.doFilter(request, response);
```

O uso de `getSession(false)` evita a criação indevida de novas sessões para usuários não autenticados, prática recomendada para reduzir consumo de memória no servidor.

4.4 Encerramento e expiração da sessão

O logout é implementado de forma explícita, invalidando a sessão no servidor com `session.invalidate()`, o que remove todos os atributos armazenados e sinaliza ao container que o Session ID não deve mais ser aceito. Complementarmente, define-se um tempo de expiração por inatividade (`session.setMaxInactiveInterval()`), garantindo que sessões abandonadas sejam liberadas automaticamente — medida de segurança discutida por Basham no contexto de proteção contra sequestro de sessão (session hijacking).

4.5 Considerações de segurança do cookie

- **HttpOnly:** impede o acesso ao cookie via JavaScript no cliente, mitigando ataques de XSS.
- **Secure:** restringe o envio do cookie a conexões HTTPS, evitando interceptação em canais não criptografados.
- **Regeneração do Session ID após login** (`session.changeSessionId()`): previne ataques de fixação de sessão (session fixation).

5. Parecer Técnico

A adoção do padrão MVC combinada ao gerenciamento de sessão via `HttpSession` e cookie `JSESSIONID` é a solução tecnicamente mais adequada para a reestruturação proposta, pelos seguintes motivos:

- **Desacoplamento e manutenibilidade:** ao isolar a lógica de autenticação no Model e o controle de fluxo no Controller, a View permanece livre de lógica de negócio, reduzindo o acoplamento identificado como principal falha da aplicação legado.
- **Aderência ao padrão da plataforma:** o uso de `HttpSession` aproveita o mecanismo nativo do container Servlet, sem necessidade de soluções customizadas de rastreamento de estado, conforme recomendado por Basham e Kurniawan como abordagem padrão de mercado.
- **Escalabilidade compatível com o cenário atual:** para uma aplicação monolítica single-server, a sessão em memória no container é suficiente; em evolução futura para múltiplas instâncias, recomenda-se sticky sessions ou persistência de sessão compartilhada (ex.: Redis).
- **Segurança adequada ao domínio:** a combinação de cookie `HttpOnly/Secure`, expiração por inatividade e regeneração do identificador após login atende aos requisitos mínimos de proteção contra os principais vetores de ataque a sessões web (fixation, hijacking e XSS).

Conclui-se que a arquitetura especificada atende ao objetivo de separar corretamente as responsabilidades entre View, Controller e Model, ao mesmo tempo em que resolve, de forma padronizada e segura, a limitação de estado inerente ao protocolo HTTP.

Referências bibliográficas: BASHAM, Bryan; SIERRA, Kathy; BATES, Bert. *Head First Servlets and JSP*. O'Reilly Media. · KURNIAWAN, Budi. *Servlet & JSP: A Tutorial*. Brainy Software.